CO4: ASSESSMENT TOOL 3: Reverse Engineering Task

Name: K Sumanth Kumar Reddy

Reg No: 192425222

SET 1

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.

2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.

3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance

requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.

5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

1. Big Data Platform for Website Clicks, Mobile Interactions and Customer Transactions

A platform collecting billions of website clicks, mobile interactions, and customer transactions requires a distributed big-data architecture capable of handling high volume, velocity, and variety.

Probable Distributed Storage System

The platform would most likely use **Hadoop Distributed File System (HDFS)** or a cloud object store such as Amazon S3, Azure Data Lake Storage, or Google Cloud Storage. HDFS can distribute large files across multiple machines and provide fault tolerance through data replication.

Structured transaction data may be stored in relational databases such as MySQL or PostgreSQL, while semi-structured clickstream and mobile-event data may be stored in **MongoDB, Cassandra, HBase, or a data lake**.

Data Ingestion Framework

High-volume data can be collected using **Apache Kafka**. Website clicks, mobile events, and transaction events are converted into messages and published to Kafka topics.

For batch ingestion from databases and external systems, **Apache Sqoop** or modern ETL tools can be used.

Batch Processing

Large historical datasets can be processed using **Apache Spark** or Hadoop MapReduce. Spark is particularly suitable because it supports fast distributed processing and SQL-based analytics.

Structured and Semi-Structured Data

Structured transaction data contains predefined fields such as:

- Customer ID
- Product ID
- Transaction amount
- Date and time

Semi-structured data such as clickstream logs may be stored in JSON format. These datasets can be placed in separate logical zones in a data lake and later integrated using Spark or Hive.

Indexes and partitions can be created using customer ID, date, region, product ID, or transaction type to make analytical queries faster.

Stream Processing

Real-time processing can be implemented using **Apache Kafka Streams, Apache Flink, or Spark Structured Streaming**. These systems can detect user activities and calculate real-time metrics such as active users, purchases, and conversion rates.

Scalability and Partitioning

The system can scale horizontally by adding more servers. Kafka topics can be divided into partitions, while HDFS files can be distributed across DataNodes.

Data can be partitioned by:

- Date
- Region
- Customer ID
- Event type

Replication ensures that data remains available even when a server fails.

Dashboard and BI Connection

Processed data can be loaded into an analytical warehouse such as **Hive, Snowflake, BigQuery, or Amazon Redshift**. Business intelligence tools such as Power BI or Tableau can connect to this analytics layer.

The final architecture is:

Web/Mobile/Transactions → Kafka → HDFS/Data Lake → Spark/Hive → Data Warehouse → BI Dashboard

Thus, the architecture provides scalable storage, real-time processing, historical analytics, and business intelligence.

2. E-Commerce Platform for Cart, Checkout, Product Views and Payments

An e-commerce platform requires real-time processing because customer actions must immediately influence recommendations, offers, and personalization.

Event Streaming

The probable event-streaming system is **Apache Kafka**. Events such as product views, cart additions, checkout attempts, payments, and abandoned carts can be published to different Kafka topics.

For example:

Product View → Kafka → Stream Processor → Customer Profile

Customer Profiling Database

Customer profiles can be stored in databases such as **Cassandra, MongoDB, Redis, or DynamoDB**. These systems provide fast access to customer preferences and recent activity.

Redis can be used as a cache for frequently accessed information.

Recommendation Workflow

The recommendation pipeline may contain:

1. Collect customer events.
2. Store clickstream and transaction history.
3. Generate customer features.
4. Train recommendation models.
5. Generate product recommendations.
6. Store recommendations in a fast-access database.
7. Display recommendations on the website.

Machine-learning techniques such as collaborative filtering, content-based filtering, and neural recommendation models may be used.

Synchronizing Data

Session information, browsing logs, and transaction records can be connected using a unique **Customer ID or Session ID**.

For example:

Customer ID → Product View → Cart Addition → Payment → Purchase History

Kafka provides ordered event streams, while stream-processing systems can combine events and generate real-time customer profiles.

Caching and Query Engines

Redis can cache:

- Product details
- Customer preferences
- Shopping cart information
- Recommendations

Distributed query engines such as **Presto/Trino or Spark SQL** can query large datasets stored in a data lake.

Combining Structured and Semi-Structured Data

Structured order information can come from an SQL database, while browsing information may be JSON-based.

Spark can join both datasets using Customer ID, Product ID, and timestamps.

Scalability and Fault Tolerance

Kafka partitions allow horizontal scaling. Replication protects messages from broker failures.

The storage layer can also replicate data across nodes. Spark can automatically recover failed tasks, while load balancing distributes traffic across servers.

Dashboard and KPI Monitoring

Important KPIs include:

- Conversion rate
- Cart abandonment rate
- Average order value
- Number of orders
- Revenue
- Product views
- Recommendation click-through rate

Data can be aggregated using Spark/Flink and loaded into a data warehouse. Power BI, Tableau, or Grafana can then provide dashboards.

The architecture can be summarized as:

Customer Events → Kafka → Flink/Spark → Customer Profile + ML → Redis → Website

and:

Historical Data → Data Lake → Spark/Trino → Data Warehouse → KPI Dashboard

3. Big Data Healthcare System

A healthcare platform handling patient histories, diagnostic images, wearable streams, and prescription information requires a hybrid architecture because it processes structured, semi-structured, and unstructured data.

Hybrid Database Architecture

Structured information such as patient demographics, prescriptions, appointments, and laboratory results can be stored in **PostgreSQL, MySQL, or Oracle**.

Unstructured information such as medical images, scanned reports, and documents can be stored in a **data lake or object storage**.

Large medical images can use specialized medical imaging systems such as **PACS**, while metadata can be stored in databases.

Wearable sensor data can be stored using time-series or distributed databases.

Data Integration

Hospitals, laboratories, pharmacies, and insurance systems can be integrated using healthcare interoperability standards such as **HL7 and FHIR**.

ETL tools such as **Apache NiFi, Kafka Connect, Talend, or cloud integration services** can collect and transform information from different systems.

Streaming and Machine Learning

Wearable devices continuously generate heart rate, oxygen level, activity, and other measurements.

Kafka can ingest this stream, while **Apache Flink or Spark Structured Streaming** can analyze it in real time.

Machine-learning models can then predict:

- Disease risk
- Abnormal vital signs
- Patient deterioration
- Readmission probability
- Treatment outcomes

Privacy and Security

Healthcare data requires strong security controls.

The platform would likely use:

- Encryption at rest
- Encryption during transmission
- Role-Based Access Control
- Multi-factor authentication
- Data masking
- Audit logs
- Patient consent management

Sensitive patient information should only be accessible to authorized users.

Compliance

The architecture should support applicable healthcare regulations and privacy requirements. Access to patient records should be logged so that administrators can identify who accessed or modified information.

Analytics Pipeline

Hospital/Lab/Wearables → Kafka/NiFi → Data Lake + Database → Spark/Flink → ML Models → Clinical Dashboard

Historical patient data can be processed using Spark to identify population-level disease trends.

Clinical Decision Support

The analytics layer can provide doctors with risk scores, abnormal-result alerts, and predictive information. Population health researchers can use aggregated and de-identified data to study disease patterns.

Therefore, the hybrid architecture supports both real-time patient monitoring and large-scale healthcare research.

4. Smart City Big Data Platform

A smart-city platform receives continuous information from traffic sensors, CCTV systems, weather stations, and public transport systems. This requires edge computing, real-time streaming, distributed storage, and geospatial analytics.

Edge Computing

Traffic cameras and sensors can perform initial processing at the edge.

For example, instead of sending every raw CCTV frame to the central system, edge devices can extract metadata such as:

- Vehicle count
- Vehicle type
- Traffic density
- Location
- Timestamp

This reduces network bandwidth and latency.

Distributed Messaging

Apache Kafka or another distributed messaging system can collect sensor events.

Different Kafka topics may represent:

- Traffic
- Weather
- Public transport
- CCTV metadata

Centralized Storage

Historical data can be stored in a distributed data lake such as HDFS or cloud object storage.

Time-series information can be stored in systems such as **InfluxDB or TimescaleDB**, while geospatial information can be stored using **PostGIS** or other spatial databases.

Geospatial Analytics

Location-based analysis can identify:

- Congested roads
- Accident-prone areas
- Bus delays
- Traffic patterns
- High-density zones

Spark can process large historical datasets, while Flink can process streaming traffic events.

Congestion Prediction

Real-time traffic data can be combined with weather, historical traffic, and public transport information.

A machine-learning model can predict:

Current Traffic + Weather + Historical Patterns → Future Traffic Congestion

Batch and Real-Time Analytics

Real-time analytics provides immediate alerts and traffic-control information.

Batch analytics can analyze months or years of data for:

- Road planning
- Public transport optimization
- Infrastructure development

- Traffic pattern analysis

Security

Security measures may include:

- Encryption
- Authentication
- Role-Based Access Control
- Network segmentation
- Secure APIs
- Audit logging

CCTV and citizen-related information should have stricter access controls.

Visualization

Live dashboards can be built using **Grafana, Power BI, Tableau, or Kibana**.

The dashboard can display:

- Live traffic density
- Congestion alerts
- Bus locations
- Weather conditions
- Road incidents

The architecture is:

Sensors/CCTV → Edge Processing → Kafka → Flink/Spark → Data Lake/Databases → ML → Live Dashboard

This architecture enables both real-time operational control and long-term urban planning.

5. Healthcare Analytics System

A healthcare analytics system combining patient records, wearable data, laboratory reports, and appointment histories requires a secure hybrid big-data architecture.

Hybrid Storage Architecture

Structured information such as patient records, appointments, and laboratory results can be stored in relational databases such as PostgreSQL or MySQL.

Semi-structured wearable data can be stored in JSON-compatible databases or a data lake.

Unstructured medical reports and documents can be stored in distributed object storage.

A possible architecture is:

SQL Database + NoSQL Database + Data Lake + Time-Series Database

ETL and Interoperability

ETL pipelines can be implemented using **Apache NiFi, Kafka, Spark, or cloud ETL services**.

Healthcare interoperability can use **HL7/FHIR** to exchange patient information between hospitals and clinics.

The pipeline can be:

Hospital/Clinic → ETL → Validation → Transformation → Data Lake/Warehouse

Real-Time Monitoring

Wearable devices continuously send patient readings.

Kafka can ingest these readings, while Flink or Spark Streaming can detect abnormal patterns.

For example:

Wearable Reading → Kafka → Stream Processing → Risk Detection → Doctor Alert

Predictive Health Analytics

Machine-learning models can analyze patient history, laboratory values, wearable readings, and appointment information.

Possible predictions include:

- Disease risk
- Readmission risk
- Patient deterioration
- Treatment response
- Emergency risk

Distributed Processing

Healthcare organizations may have millions of patient records. **Apache Spark** can distribute processing across multiple machines.

This enables population-level studies such as analyzing disease trends across regions or age groups.

Encryption and Compliance

Healthcare information requires strong governance.

The platform can implement:

- Encryption at rest
- TLS encryption in transit
- Role-Based Access Control
- Authentication
- Data masking
- Audit trails
- Backup and disaster recovery
- Patient consent controls

Machine Learning

Machine-learning models can produce diagnosis risk scores using patient features.

For example:

Patient History + Lab Results + Wearable Data → ML Model → Risk Score

Doctors can use these scores as decision-support information. Treatment recommendation systems can use patient history and clinical information to suggest potentially suitable treatment options, while final decisions remain with qualified healthcare professionals.

Overall Architecture

Patient Records + Wearables + Labs + Appointments
↓
ETL / Kafka / FHIR
↓
Data Lake + SQL + NoSQL + Time-Series Storage
↓
Spark / Flink
↓
Machine Learning
↓
Risk Scores + Clinical Analytics
↓
Secure Healthcare Dashboard

Thus, the system provides scalable population-level analytics while supporting real-time monitoring, predictive analysis, interoperability, security, and healthcare data governance.